L9: The Arcane Sorting Chamber

Following the success of Archibald and Eleonora's datagram chess match, the Grand Council of Aethermoor was inspired. Mages across the realm had long struggled with unreliable spell-scroll couriers. The Council's solution: an Arcane Sorting Chamber — a UDP server that accepts parcel requests from registered wizards, sorts them by urgency, and dispatches courier familiars to deliver them.

Protocol

All datagrams are text-based, fields separated by **;**, terminated by **\n**, maximum size MAX_MSG_LEN bytes:

```
&lt;TYPE&gt;;&lt;field1&gt;;&lt;field2&gt;;...\n
```

Type	Format	Description
REG	REG;<name>\n	Register (name: max MAX_NAME_LEN chars, alphanumeric + _)
SEND	SEND;<priority>;<contents>;<r1>;<r2>;...\n	Send parcel (priority 1-5, contents max MAX_CONTENTS_LEN)
FETCH	FETCH\n	Collect a parcel from own mailbox
STATUS	STATUS\n	Request queue size and pebble count
QUIT	QUIT\n	Deregister and leave

> *All datagram parsing must use **strtok_r** — the number of recipients in SEND is variable. Since courier threads also call the same parsing routine, strtok is unsafe due to its hidden static state.*

```
./sop-chamber <port>
```

Constants

```
#define MAX_MSG_LEN          256
#define MAX_NAME_LEN         32
#define MAX_CONTENTS_LEN     64
#define MAX_WIZARDS          10
#define MAX_RECIPIENTS       5
#define COURIER_COUNT        3
#define COURIER_DELAY_MS     500
#define MAX_DISPATCH_QUEUE   20
#define MAX_TOTAL_PARCELS    30
#define STARTING_PEBBLES     15
#define JUDGE_INTERVAL_S     2
#define MAX_MESSAGES         20
```

Stage 1 (6 pts)

The server listens for UDP datagrams on the given port. Parse every incoming datagram with strtok_r and print to stdout:

```
[REG] Welcome to the Chamber, <name>!
[SEND] Parcel to <r1>, <r2>, ... (priority <p>): <contents>
[FETCH] Fetch request received
[STATUS] Status request received
[QUIT] Goodbye!
```

No persistent state in this stage. On any malformed datagram (unknown type, missing fields, priority outside 1-5, name or contents too long, more than MAX_RECIPIENTS recipients), print an error but continue operation. The program ends after processing MAX_MESSAGES = 20 valid messages.

> *Hint: Strip the trailing \n before parsing. Call strtok_r in a loop with ";" as delimiter.*

Stage 2 (7 pts)

COURIER_COUNT courier familiar threads process deliveries asynchronously. Maintains a fixed-size array of up to MAX_WIZARDS registered wizards (name + UDP address) and a FIFO mailbox per wizard. Global dispatch queue — circular buffer of capacity MAX_DISPATCH_QUEUE:

```
typedef struct {
    dispatch_entry_t buf[MAX_DISPATCH_QUEUE];
    int head, tail, count;
    pthread_mutex_t mutex;
    pthread_cond_t not_empty;
} dispatch_queue_t;
```

Enqueue writes to buf[tail % MAX_DISPATCH_QUEUE] and increments tail; dequeue reads from buf[head % MAX_DISPATCH_QUEUE] and increments head. Courier threads wait on not_empty — active waiting is forbidden.
When a courier picks up an entry it: (1) parses with strtok_r, (2) waits COURIER_DELAY_MS ms, (3) appends parcel to each recipient's FIFO mailbox, (4) prints:

```
[Courier] Delivered to <recipient>: <contents> (priority <p>)
```

If dispatch queue is full: **[ERROR] Dispatch queue full — parcel dropped!**

Stage 3 (6 pts)

Registration: Before 2 wizards register, only REG is accepted. After game starts, unregistered addresses are discarded.
Mailbox upgrade: Replace FIFO with sorted linked list ordered by priority ascending (1 = most urgent); ties broken by arrival order. FETCH returns highest-priority parcel.
Pebble economy: Each wizard starts with STARTING_PEBBLES = 15.

Priority	Cost per recipient
1	4 pebbles
2	3 pebbles
3	2 pebbles
4-5	1 pebble

Total cost = N_recipients x cost. If insufficient: send 2-byte response E + pebbles as uint8_t. On successful SEND: send K + remaining pebbles. On FETCH: send PARCEL;<sender>;<priority>;<contents>\n. On STATUS: send STATUS;<n>;<pebbles>\n.
Use a POSIX counting semaphore initialised to MAX_TOTAL_PARCELS = 30. Couriers call sem_trywait before inserting. FETCH calls sem_post after removing.

Stage 4 (5 pts)

A judge thread starts when the second wizard registers. Every JUDGE_INTERVAL_S = 2 seconds it adjusts each wizard's pebbles by **DK = floor(n/2) - 1** where n is parcels in mailbox, sends 2-byte datagram P + pebbles, and prints:

```
=== Chamber Status ===
  <name_1>: <k1> pebble(s), <n1> parcel(s) waiting
  <name_2>: <k2> pebble(s), <n2> parcel(s) waiting
======================
```

If any wizard reaches 0 or below, their opponent wins. Judge prints:

```
-= Congratulations, <name> — you are the Grand Archmage! =-
```

Sends W to winner and L to loser (1-byte datagrams).
Signal handling: Before creating any thread, block SIGINT and SIGTERM using pthread_sigmask. Main loop uses sigtimedwait alongside recvfrom. On signal: set stop_flag, broadcast on dispatch queue condition variable, join all threads, free all mailboxes, destroy mutexes and semaphore, close socket, print:

```
[Chamber] Shutting down. Final state:
  <name_1>: <k1> pebble(s), <n1> parcel(s) unread
  <name_2>: <k2> pebble(s), <n2> parcel(s) unread
[Chamber] Goodbye!
```

Error Messages Reference

Condition	Message

Datagram too long	[ERROR] Datagram exceeds MAX_MSG_LEN, discarded.
Unknown type	[ERROR] Unknown message type: '<X>'.
Missing/malformed fields	[ERROR] Malformed <TYPE> message.
Priority out of range	[ERROR] Priority must be between 1 and 5.
Too many recipients	[ERROR] Too many recipients (max MAX_RECIPIENTS).
Invalid name	[ERROR] Invalid wizard name: '<name>'.
Unknown recipient	[ERROR] Unknown recipient: <name>.
Insufficient pebbles	[ERROR] <name> cannot afford priority <p> to <N> recipient(s) (has <k> pebble(s)).
Dispatch queue full	[ERROR] Dispatch queue full — parcel dropped!
Chamber at capacity	[ERROR] Chamber at capacity — parcel dropped!
Fetch on empty mailbox	[ERROR] <name>'s mailbox is empty.
Before game start	[ERROR] Chamber not yet open — awaiting second wizard.
Unregistered address	[ERROR] Unregistered address, message discarded.